

SQLChange: An Agentic Reasoning Pipeline for SQL Query Optimization

Ahmed Khan • Fahim Islam • Dev Rathod • Arnav Akula • Devarchith Parashara Batchu

ECS 189G — Large Foundation Models: Final Project
University of California, Davis

June 2026

Problem & Motivation

Problem

- ▶ SQL optimization is manual and requires expertise
- ▶ Performance bottlenecks are hard to identify before execution
- ▶ Existing tools lack reliable verification of correctness and speedups

Motivation

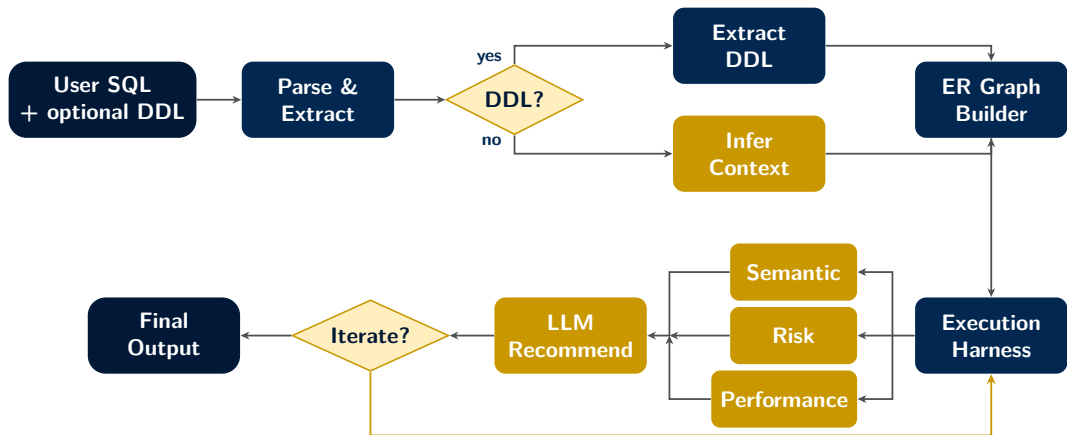
- ▶ Petabyte-scale data makes inefficient queries expensive
- ▶ Privacy constraints limit the use of external AI tools
- ▶ Engineers need an end-to-end agent for optimization, validation, and explanation

Our Question

Can an AI agent optimize SQL queries for **performance** while preserving **correctness**?



Agentic Reasoning Pipeline

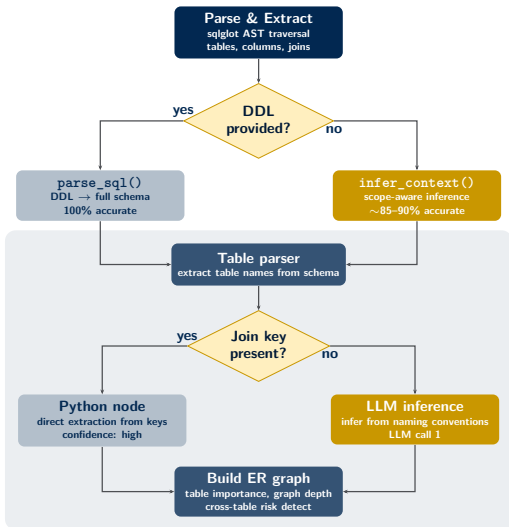


DDL = Data Definition Language (CREATE TABLE, schema definitions)

Input / Output

Deterministic

AI / LLM



ER Graph Construction

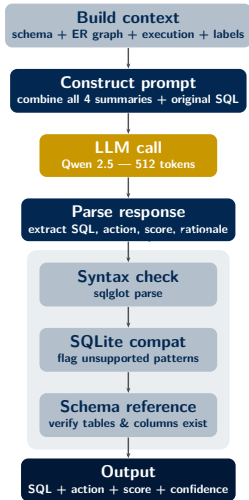
► Table importance hierarchy:

- *Root* — never a join target (changes cascade everywhere)
- *Intermediate* — both source and target
- *Leaf* — only a target (safest to modify)

► Cross-table risk signals: graph depth, join-WHERE overlap, dependency fan-out

- Feeds context to recommendation engine and risk labeler downstream

Recommendation Engine

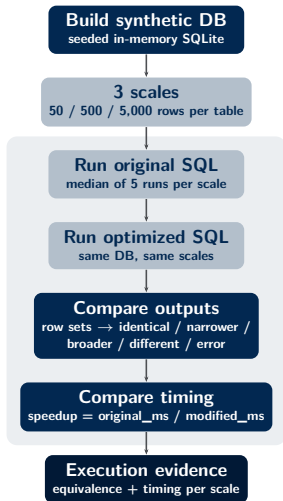


LLM-Powered Optimization

- ▶ LLM receives **full upstream context**: schema summary, ER graph hierarchy, execution evidence, and prior iteration labels
- ▶ Outputs: optimized SQL, action type (optimize / rewrite / keep_original), confidence (1–10), rationale
- ▶ On later iterations: sees why the last attempt scored poorly and adjusts (conservative if risk ≥ 7)

Three-Layer Validation

- ▶ **Syntax** — sqlglot must parse output as valid SQL
- ▶ **SQLite compat** — flags DATE_SUB, INTERVAL, IF(), ILIKE, RIGHT JOIN; suggests equivalents
- ▶ **Schema ref** — every table/column must exist in schema; validation fails → confidence downgraded



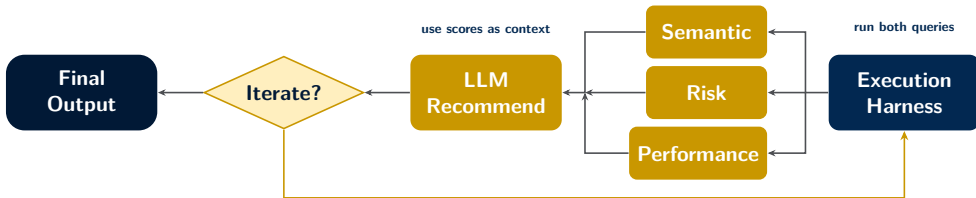
Synthetic Database

- ▶ **Deterministic:** seeded by query hash, reproducible across runs
- ▶ 75% join match rate with realistic boundary values for WHERE clauses
- ▶ SQL normalized for SQLite (DATE_SUB → strftime, true → 1)

Reasoning Labels

- ▶ **Semantic** — deterministic classification from execution output (identical / narrower / broader / different); LLM refines only when confidence is low
- ▶ **Performance** (1–10) — multi-scale timing; 8–10 = major speedup, 1–2 = degradation
- ▶ **Risk** (1–10) — table importance hierarchy, join depth, cross-table WHERE, row-count mismatches

Iterative Refinement



How It Works

- ▶ Harness executes original + optimized SQL on synthetic data at 3 scales
- ▶ Labelers score: deterministic rules first, LLM only if confidence is low
- ▶ Scores feed back to LLM — it sees *why* the last attempt failed
- ▶ High risk → conservative mode; wrong results → prioritize correctness

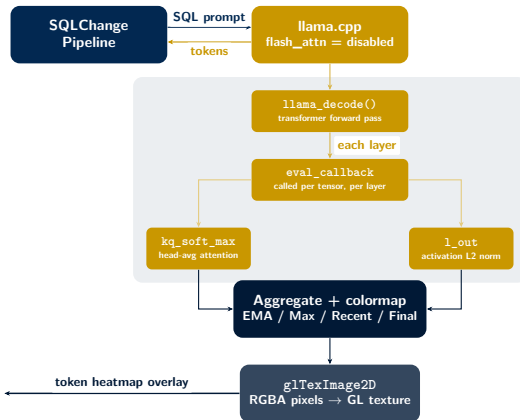
Stop Conditions

- ▶ $\text{Perf} \geq 8$ **and** $\text{risk} \leq 3$
- ▶ LLM recommends `keep_original`
- ▶ SQL fails validation
- ▶ Max iterations reached (default: 3)

Attention Heatmap

```
[attn OK] gen_steps=178 latest_layers=48 ctx_map=48
Generating... (178 tokens)
Max 3.5 Contrast
user
You are a senior database optimization engineer. You are given
full pipeline analysis from execution benchmarks, risk assessment,
semantic analysis, and schema context. Your job is to produce
that improves performance and/or reduces risk while preserving
Original SQL:
SELECT DISTINCT e.*, d.dept_name,
  (SELECT COUNT(*)
   FROM assignments a2
   WHERE a2.employee_id = e.id) AS project_count,
  (SELECT SUM(hours_worked)
   FROM assignments a3
   WHERE a3.employee_id = e.id) AS total_hours
FROM employees e
LEFT JOIN departments d ON e.department_id = d.id
LEFT JOIN assignments a ON a.employee_id = e.id
LEFT JOIN projects p ON p.id = a.project_id
WHERE e.is_active = 1
AND e.id NOT IN (
  SELECT employee_id FROM assignments
  WHERE project_id IN (
    SELECT id FROM projects WHERE status = 'cancelled'
  )
)
AND UPPER(d.dept_name) != 'TEMP'
AND e.salary > (
  SELECT AVG(salary) FROM employees
  WHERE department_id = e.department_id
)
ORDER BY e.salary DESC
```

Token-level attention overlay on the prompt context during SQL optimization.



Evaluation Setup

- ▶ **100 queries** built from gretelai/synthetic_text_to_sql, excluding development set
- ▶ Stratified: 34 simple, 37 moderate, 29 complex
- ▶ **20 degraded queries** — programmatically injected inefficiencies (unnecessary subqueries, redundant joins)
- ▶ **20 no-DDL queries** — schema must be inferred
- ▶ Each query run through full pipeline with 5-minute timeout, up to 3 iterations

Metrics

- ▶ **Semantic equivalence** — does the optimized query return the same results?
- ▶ **Performance score** (1–10) — speedup across 3 scales
- ▶ **Risk score** (1–10) — safety of the change
- ▶ **Validity** — does the recommendation parse and match the schema?
- ▶ **Iteration depth** — passes before convergence
- ▶ **Wall-clock time** — end-to-end latency

Complexity Tier	Queries	Sem. Equiv.	Avg. Perf.	Avg. Risk	Avg. Iters
Simple (basic SQL, single join)	34	79%	6.6	3.6	1.2
Moderate (aggregation, subqueries)	37	73%	6.8	4.0	1.7
Complex (multi-join, window, set)	29	52%	7.2	3.8	1.8
Overall (100 queries)	100	68%	6.9	3.8	1.6

Key Findings

- ▶ **Reliable:** only 1 parse error in 100 queries
- ▶ **Conservative:** 55% kept original — avoids unnecessary optimization. Optimization rate scales with complexity (12% simple → 51% moderate → 66% complex)
- ▶ **Anti-pattern detection:** 75% of degraded queries correctly fixed — SELECT * and unnecessary subqueries reliably caught
- ▶ **DDL-robust:** equivalence similar for DDL vs. no-DDL — inference path performs just as well

Limitations

- ▶ **Complex queries:** only 52% equivalence — 14 of 22 non-equivalent results were complex queries; joins and window functions cause subtle result-set changes
- ▶ **Iteration penalty:** single-iteration queries scored 7.5 avg perf vs. 5.6 for multi-iteration — harder queries that need more passes are also harder to optimize well
- ▶ **Risk is conservative:** only 15% of optimizations flagged high-risk, appropriate for production SQL

What We Built

- ▶ End-to-end agentic pipeline that *generates* optimized SQL — the proposal planned comparison only; we went further
- ▶ Execution-verified: synthetic DB testing catches silent result-set changes that LLM-only approaches miss
- ▶ Iterative feedback loop — labels feed back to the LLM for progressive improvement
- ▶ Schema inference from queries alone — works without DDL, which the proposal assumed would always be available
- ▶ Runs fully local via Ollama — no data leaves the machine. Evaluation on Qwen2.5:7b

Limitations & Future Work

- ▶ SQLite only — future: multi-dialect support (PostgreSQL, MySQL)
- ▶ 110 queries vs. the proposed 250–300 pairs — future: Spider/BIRD benchmarks and real GitHub commit pairs
- ▶ No baseline comparison — future: zero-shot LLM vs. structured prompt
- ▶ Scales limited to 5k rows — future: real database connections for production-scale testing
- ▶ Local 7B model — future: larger or fine-tuned models for better output
- ▶ RAG for query-plan-aware optimization (original reach goal)

Thank You

Questions?